

FOLIO

Library Management System Testing Report

An evidence-based assessment of the React frontend, Spring Boot backend, MySQL persistence, JUnit suite, OOP design, and local browser behavior.

99 / 99

verified

BUILD
PASS

verified

MYSQL
PASS

verified

1 HIGH

verified

Executive verdict

The backend build and all 99 JUnit tests passed against a live MySQL 9.7.2 test database. The Vite production build completed successfully, and browser checks confirmed navigation, section-specific themes, search filtering, and zero console warnings/errors. The principal integration gap is that the React frontend is a browser-local demo: it uses localStorage and does not call the Spring REST API. Consequently, backend functionality is verified, but the complete UI-to-Spring-to-MySQL workflow is not implemented.

	Result	Details
Backend compile and test	PASS	Maven clean test; 99 tests, 0 failures/errors/skips
MySQL integration	PASS	Hikari opened com.mysql.cj.jdbc.ConnectionImpl; HTTP and service tests passed
Frontend production build	PASS	Vite 6.4.3; 1,571 modules; completed in 34.70 s

Measurement	Result	Evidence
Browser smoke test	PASS	Overview, Books, Readers and Loans rendered; search filter worked; console clean
End-to-end UI/backend integration	FAIL	No fetch/axios/API usage in frontend; state is persisted with localStorage

1. Scope and execution evidence

The supplied testing brief was treated as requirements context. Results below are based on commands and browser interactions executed on the current project state; no result was inferred as PASS solely from source inspection.

Commands executed

```
cd frontend
npm run build

cd backend
mvnw.cmd clean test

mysql -uroot -D librarydb_test -e "SELECT ..."
```

Component name	Observed output
Frontend	vite v6.4.3; 1,571 modules transformed; dist generated; 34.70 s
Backend	BUILD SUCCESS; total time 54.846 s
JUnit	Tests run: 99, Failures: 0, Errors: 0, Skipped: 0
Database	MySQL Community Server 9.7.2; live connection on localhost:3306
Browser	All four main sections rendered; Atomic search excluded Sapiens; no console warnings/errors

Environment

Component	Observed value
Operating system	Windows 11
Java runtime	Oracle Java 26.0.2; Maven compiler release target 21
Build tool	Apache Maven 3.9.9 via Maven Wrapper
Backend	Spring Boot 3.5.16; embedded Tomcat 10.1.55
Database	MySQL Community Server 9.7.2; isolated librarydb_test
Frontend	React 18.3.1; Vite 6.4.3; Node 25.8.1; npm 11.11.0
Revision	Git commit 02f96fd

2. Automated test results

JUnit summary

Test Class	Tests	Passed	Failed	Errors	Time
BookUnitTest	11	0	0	0	0.218 s
FineCalculatorUnitTest	11	0	0	0	0.060 s
LibrarianUnitTest	5	0	0	0	0.019 s
LibraryIntegrationTest	17	0	0	0	23.95 s
LibraryServiceTest	48	0	0	0	1.454 s
LoanRecordUnitTest	3	0	0	0	0.021 s
UserUnitTest	4	0	0	0	0.021 s
TOTAL	99	0	0	0	100% pass

Executed JUnit total: 99 | Passed: 99 | Failed: 0 | Errors: 0 | Skipped: 0 | Pass percentage: 100%

Database execution evidence

Spring Boot activated the test profile, initialized HikariCP, and opened a real `com.mysql.cj.jdbc.ConnectionImpl`. The test classes reset and seed the isolated test database before execution.

Table	State	Notes
books	15 rows: 12 available, 3 borrowed	Service tests created and exercised additional records
users	6 rows	Seed users plus service-test records
loan_history	7 rows; recorded fines total 0	Return workflows wrote history records

Note: this is a post-test snapshot, not production data. The suite intentionally mutates and resets only `librarydb_test`. No production database was used.

3. Test-case table

The table mixes executed automated tests, executed browser smoke tests, and one source-backed integration check. TC-25 fails because the required connection layer is absent.

TC-01	Build	Backend clean build	mvnw.cmd clean test	Compile and execute tests	Build success	PASS
TC-02	Build	Frontend production build	npm run build	Generate dist bundle	Bundle generated	PASS
TC-03	Book	Create valid book	JUnit constructor test	Fields/status initialized	Matched	PASS
TC-04	User	Create valid user	JUnit constructor test	Identity fields initialized	Matched	PASS
TC-05	OOP	Librarian inheritance	Librarian as User	Runtime inheritance/polymorphism	Confirmed	PASS
TC-06	Book	Checkout state transition	Book.checkout	Borrow fields and due date set	Matched	PASS
TC-07	Book	Return state transition	Book.returnBook	Loan fields cleared	Matched	PASS
TC-08	Fine	Overdue calculation	Three days overdue	Fine = INR 30	INR 30	PASS
TC-09	Books	Add valid book	Service/API test	Count increases	Increased	PASS
TC-10	Books	Reject blank book	Blank title/author	Validation error	Rejected	PASS
TC-11	Books	Update book	PUT valid fields	Persist changes	Persisted	PASS
TC-12	Books	Search catalogue	Title/author/ISBN/genre	Matching results	Matched	PASS
TC-13	Readers	Register/update reader	Valid payloads	Persist user	Persisted	PASS
TC-14	Borrow	Borrow available book	Valid book and reader	Borrow succeeds	Succeeded	PASS
TC-15	Borrow	Reject unavailable book	Borrow same title twice	Rejected	Rejected	PASS
TC-16	Borrow	Enforce three-book limit	Attempt fourth loan	Rejected with limit error	Rejected	PASS
TC-17	Renew	Prevent second renewal	Renew already-renewed book	Rejected	Rejected	PASS
TC-18	Hold	Hold blocks renewal	Renew held title	Rejected	Rejected	PASS
TC-19	Return	Return and write history	POST return endpoint	Available + history row	Confirmed	PASS
TC-20	API	Unknown endpoint	GET /api/nope	HTTP 404	HTTP 404	PASS
TC-21	UI	Navigate all sections	Overview/Books/Readers/Loans	Correct section/page theme	All rendered	PASS
TC-22	UI	Search filtering	Search Atomic	Atomic shown; Sapiens hidden	Matched	PASS
TC-23	UI	Console health	Read warning/error log	No warnings/errors	None observed	PASS
TC-24	DB	Live MySQL connection	Test profile on port 3306	JDBC connection opens	Opened	PASS

Test ID	Module	Scenario	Input / Action	Expected	Actual	Status
TC-25	Integration	UI to Spring/MySQL	Inspect network code	Frontend calls /api	Uses localStorage only	FAIL

4. Defect report

No defect below was fixed during this assessment; the objective was testing and reporting. Severity reflects impact in a deployed system, with the project's academic/demo context noted.

Defect ID	Severity	Component	Expected Behavior	Actual Behavior	Test Case	Resolution
BUG-01	High	Frontend integration	React never calls the REST API; data lives in localStorage.	Search frontend/src/main.jsx for fetch, axios or /api; only localStorage is present.	Expected UI -> Spring -> MySQL. Actual browser-only demo state.	Create an API client and replace local mutations with controller calls.
BUG-02	Medium	Books / schema	Duplicate ISBN values are accepted; isbn has no UNIQUE constraint.	Add two books with the same nonblank ISBN.	Expected duplicate rejection. Actual both inserts are allowed.	Add UNIQUE(isbn), map duplicate-key errors, and add a test.
BUG-03	Medium	Security	No authentication or authorization protects librarian actions.	Call add/delete endpoints without credentials.	Expected restricted admin operations. Actual endpoints are public.	Add Spring Security and role-based authorization before production use.
BUG-04	Low	Controller / CORS	Cross-origin access is allowed from every origin.	Inspect @CrossOrigin(origins = "*").	Expected trusted origin allow-list. Actual wildcard.	Configure environment-specific allowed origins.
BUG-05	Low	Validation	ISBN is checked only for blank input, not valid ISBN-10/13 structure.	Submit isbn=not-an-isbn.	Expected format error. Actual accepted.	Implement checksum-aware ISBN validation.
BUG-06	Low	Service API	Create methods throw while update methods return error strings.	Compare addBook invalid input with updateBook invalid input.	Expected consistent error contract. Actual mixed conventions.	Use domain exceptions and centralized API error mapping.
BUG-07	Low	OOP model	User does not hold a borrowed-books list; the relationship is queried from books.	Inspect User.java and Library.getBooksBorrowedByUser.	Expected literal requirement field. Actual computed relationship.	Document the relational design or expose a read-only collection.

5. OOP, security, and reliability review

OOP design

Encapsulation	Implemented	Private model fields; getters/setters; Book methods control checkout, return, renewal and hold state
Inheritance	Implemented	Librarian extends User
Polymorphism	Implemented	Librarian overrides toString; verified through a User reference in JUnit
Abstraction	Partial	Controller delegates to Library service, but no interface or abstract class is used
Composition	Implemented	Library composes JdbcTemplate and maps relational rows into domain models

Security

SQL injection resistance	PASS (inspection)	JdbcTemplate statements use placeholders rather than concatenated user input
Authentication	MISSING	No login, password or identity verification
Authorization	MISSING	Book and reader administration endpoints are unrestricted
CORS	WEAK	Wildcard origin is enabled
XSS	PARTIAL	React text rendering escapes strings by default; backend does not sanitize stored text

Performance and reliability

A sustained load test was **NOT TESTED**. Available timing evidence is limited to the build and test runners: Vite build 34.70 s; full Maven run 54.846 s; HTTP integration class 23.95 s; service class 1.454 s. These are execution times, not endpoint latency measurements. No performance claim is made beyond successful completion.

Observed warnings

The build reported unchecked/raw Map usage in LibraryIntegrationTest and future-compatibility warnings for dynamic Mockito agent loading on JDK 26. They did not fail the current build but should be cleaned up before future JDK upgrades.

6. Feature coverage

Feature	Status	Comments / Evidence
Book management	Implemented and working	Backend CRUD/search tests pass; UI is demo-local
User management	Implemented and working	Backend register/search/update/delete tests pass
Borrowing	Implemented and working	Availability and borrowing-limit tests pass
Returning	Implemented and working	Return, fine and loan-history tests pass
Renewal	Implemented and working	One-renewal and hold-blocking rules pass
Book hold	Implemented and working	Hold validation tests pass
Borrowing limit	Implemented and working	Three-book maximum enforced
Late fee	Implemented and working	INR 10/day calculation tests pass
Search	Implemented and working	Backend title/author/ISBN/genre and browser search pass
OOP	Implemented; abstraction partial	Encapsulation, inheritance and polymorphism demonstrated
MySQL integration	Implemented and working	99 tests passed over live MySQL connection
Frontend/backend integration	Incomplete	No REST API calls from React frontend
Authentication/authorization	Missing	No access-control layer

Functional browser observations

Page	Message	Page Name	Result
Overview	Good morning, Alex.	page-overview	PASS
Books	Books & editions.	page-books	PASS
Readers	Your readers.	page-users	PASS
Loans	Loans & returns.	page-loans	PASS

The browser search for 'Atomic' displayed Atomic Habits and excluded Sapiens. The browser console contained no warning or error entries during the smoke test.

7. PPT-ready Testing & Debugging slides

Slide 1 - Testing & Debugging

Methods: JUnit unit tests, Spring Boot service tests, HTTP integration tests, live MySQL database tests, frontend production build, browser smoke testing, OOP review and security inspection. Tools: JUnit 5, Spring Boot Test, TestRestTemplate, Maven Surefire, MySQL 9.7.2, Vite and browser automation.

Slide 2 - Key test cases

Add/update/search books - PASS; register/update readers - PASS; borrow available title - PASS; reject unavailable title - PASS; enforce three-book limit - PASS; reject second renewal - PASS; hold blocks renewal - PASS; return book and write history - PASS; browser search - PASS; UI-to-backend data flow - FAIL (frontend uses localStorage).

Slide 3 - Unit testing results

JUnit tests: 99 total, 99 passed, 0 failed, 0 errors, 0 skipped. Pass rate: 100%. Maven result: BUILD SUCCESS. Full run time: 54.846 seconds.

Slide 4 - Integration / database testing

HTTP request -> Spring controller -> Library service -> OOP model -> JdbcTemplate -> MySQL. Borrow validates availability/user/limit, writes borrower and due date; return calculates the fine, adds loan history and restores availability. This backend path passed on MySQL. The React UI is not yet connected to it.

Slide 5 - Bugs found & fixes

High: connect React to the REST API. Medium: enforce unique ISBN and add authentication/roles. Low: restrict CORS, validate ISBN format, standardize error handling, and document the User-to-loans model.

Slide 6 - Final testing summary

Backend: 99/99 automated tests passed on MySQL. Frontend: build and smoke tests passed. Overall conclusion: core library business logic is stable and well covered; production readiness is blocked by missing frontend/backend integration and access control.

Recommended presentation screenshots

1. JUnit result	Terminal lines showing 99 tests, zero failures/errors/skips, and BUILD SUCCESS
2. Functional UI	Overview plus one themed section such as Books, showing clay icons
3. Database evidence	MySQL query result for books/users/loan_history in librarydb_test
4. Architecture gap	Code search showing localStorage usage and absence of fetch/API calls
5. Final summary	The executive verdict on page 1 of this report

8. Final conclusion

The Folio backend is in a strong state for an academic Java OOP project: it compiles cleanly, uses a real MySQL database, demonstrates core OOP principles, and passes all 99 automated tests. The browser interface also builds and behaves correctly as a polished standalone demo.

The most important next engineering task is to replace localStorage-based frontend operations with calls to the Spring REST API. Until that is completed, the interface and backend are two separate working systems rather than one full-stack application. Authentication, ISBN uniqueness, and stricter CORS should follow before deployment beyond a controlled classroom/demo environment.

JUnit pass rate	100% (99/99)
Build status	Frontend PASS; backend PASS
Database status	MySQL integration PASS
Browser smoke test	PASS
End-to-end UI/backend	FAIL / incomplete
Critical defects	0
High defects	1
Medium defects	2
Low defects	4

Report generated from executed evidence on 27 September 2026 (Asia/Calcutta).